

华中科技大学

研究生（文献阅读与选题）报告

☐文献阅读、☒选题、☐文献阅读与选题、☐论文中期进展

标题：工具调用型大语言模型智能体故障诊断的信息表征与交互设计研究

学 号 M202570493

姓 名 陈思静

专 业 设计学

指 导 教 师 何心

院（系、所） 设计学院

填表注意事项

- 一、本表适用于攻读硕士、博士学位研究生文献阅读报告、选题报告、文献阅读与选题报告、论文中期进展报告。
- 二、以上各报告内容及要求由相关院（系、所）做具体要求。
- 三、以上各报告均须存入研究生个人学籍档案。
- 四、本表填写要求文句通顺、内容明确、字迹工整。
- 五、A4 纸正反打印

目 录

一、选题报告	1
1 选题依据	(1)
2 国内外研究综述	(3)
3 研究问题与方法	(12)
4 研究框架	(21)
5 计划进度	(22)
参考文献	23

一、选题报告

1 选题依据

1.1 研究来源

本课题来源于工具调用型大语言模型智能体的实际开发工作。智能体完成一项任务时，通常需要多次请求模型、调用外部工具并更新运行状态。任务失败后，开发者需要查阅提示、调用参数、工具返回、状态变化和错误信息，判断问题发生的环节及其对后续步骤的影响。这些材料分散在调用平台、日志、配置和代码中，增加了查找和理解的难度。

1.2 研究背景

大语言模型（large language model, LLM）通过工具调用参与外部任务执行，形成大语言模型智能体（large language model agent）。Yao 等^[1]提出的推理与行动协同方法（ReAct）将语言推理与环境操作相结合，使模型能够根据任务进展继续采取行动。Cemri 等^[2]对多智能体失败的分析以及 Chen 等^[3]对智能体框架缺陷的研究表明，故障可能涉及模型输出、工具交互、状态传递和流程控制等多个环节。开发者需要把最终症状与运行事件联系起来，比较原因解释并决定后续检查。

Epperson 等^[4]通过开发者访谈和用户研究发现，长对话检查、问题定位及运行调整存在困难，提供编辑和重执行功能后，使用者仍需判断修改位置和方式。相邻领域的研究也揭示了人的持续判断活动：van der Maden 等^[5]对十九名大语言模型产品实践者的访谈发现，评价结果与具体改进行动之间存在衔接困难；Kumar 等^[6]对开发者使用编程智能体完成仓库任务的观察，记录了持续评价、增量交互和人工干预。

本课题关注开发者从任务失败回查运行步骤、判断失败原因的过程，重点研

究已有工具下仍然存在的溯源困难。交互设计涉及运行记录如何呈现，以及开发者查找、关联和比较这些记录的操作。研究将从实际排错事件出发，选择一项关键困难，再提出设计方案并评价其作用。

1.3 研究目的及意义

本研究旨在识别开发者使用现有工具进行智能体错误溯源的困难，围绕其中一项关键困难提出交互设计方案，并检验其能否改善开发者的原因判断表现。研究结果用于总结设计建议及其适用条件。

具体目标包括：

- 1) 通过真实事件访谈和任务观察，识别开发者使用现有工具进行错误溯源时的困难，明确困难发生的条件及实际影响。
- 2) 根据经验研究确定设计问题，比较备选方案，形成能够表达和检验核心设计主张的原型。
- 3) 通过对照任务评价设计对原因判断及完成成本的影响，并结合新案例观察说明设计建议的适用范围。

1.3.1 理论意义

本研究面向交互设计领域，拟形成经验描述和设计原则两类学术成果。经验描述用于说明开发者在什么任务和信息条件下难以定位相关事件、建立事件关系、区分原因解释或决定后续检查，补充程序调试研究在自然语言记录、概率性输出和外部工具混合情境中的经验材料。

设计原则概括针对共同诊断困难的设计操作、预期作用和适用条件。Höök和 Löwgren^[7]讨论的中层知识（intermediate-level knowledge）为这一贡献定位提供了参考。原则将具体实例与可迁移的设计认识联系起来，使其他设计者能够判断其适用性并在不同界面中实施。原则由形成性研究、方案比较和评价结果共同形成，按照“问题条件—设计操作—预期作用—经验依据—适用边界”的结构表述。研究原型用于把设计主张转化为可操作方案，并在诊断任务中接受检验。

1.3.2 实践意义

本研究将形成可复现的诊断案例、研究原型和评价材料，为智能体调试工具的设计提供依据。研究结果可帮助工具设计者判断哪些信息需要被记录，哪些关系需要被呈现，以及哪些操作有助于使用者检查自己的判断。案例与评分材料还可为开发团队评价调试界面和改进运行记录提供参考。

2 国内外研究综述

本综述围绕开发者从任务失败回查运行过程、判断失败原因这一活动展开，依次讨论错误溯源的任务特点、开发者已知的困难和现有工具的解决程度，再据此确定本课题需要继续研究的问题。文献比较重点关注研究对象、任务、设计能力和实际评价结果。

2.1 工具调用型智能体的错误溯源任务

2.1.1 从任务失败回查运行过程

Yao 等^[1]提出的推理与行动协同方法将语言推理、工具操作和环境反馈组织为交替过程。智能体的一次任务由多个运行步骤共同完成，最终结果取决于模型输出、工具返回以及后续步骤对这些内容的使用。Cemri 等^[2]通过多智能体失败分析建立故障分类，涉及系统设计、智能体间协调和任务验证；Chen 等^[3]进一步研究运行框架缺陷，将失败轨迹与数据流、控制流及实现位置联系起来。这些研究说明，失败分析需要检查任务执行过程中多个环节的联系。

本课题将错误溯源界定为：开发者从失败结果出发，回查相关运行步骤，并根据运行记录判断失败原因。例如，错误的最终回答可能与检索参数、工具返回或后续步骤对返回内容的使用有关。定位一条异常记录之后，还需要说明该记录如何与最终失败相联系。该任务以原因判断为主要结果，修复与运行引导属于其后续活动。

2.1.2 运行记录与自动归因提供的基础

Fonseca 等^[8]提出的跨层任务追踪框架（X-Trace）、Sigelman 等^[9]总结的大规模追踪基础设施（Dapper），以及 Mace 等^[10]提出的动态因果监测系统（Pivot Tracing），通过标识传播、事件采集和查询支持跨组件运行分析。Zheng 等^[11]提出的系统级智能体观测工具（AgentSight）进一步关联模型交互与底层系统活动。这些工作解决了部分运行事实如何被记录和关联的问题，也说明可分析的范围取决于记录内容及其完整性。

Zhang 等^[12]围绕“哪个智能体在何时导致失败”构建自动失败归因任务，评测方法能否定位责任主体和步骤。Chen 等^[3]则关注框架缺陷的自动诊断和修复。自动方法的定位结果可以提供检查线索；开发者使用这些线索时的理解、检查和判断，需要相应的用户任务证据。多智能体归因还包含主体间责任分配问题，其结论用于单个工具调用型智能体时，需要结合具体故障条件判断。

运行追踪与自动归因已经提供数据和分析能力。交互设计需要进一步面对的任务是：使开发者能够找到与失败有关的记录，并据此判断可疑步骤是否能够解释失败。记录缺失、材料访问困难和原因解释困难涉及不同条件，需要在实际排错过程中分别识别。

2.2 开发者错误溯源中的已知困难

2.2.1 长轨迹检查与原因判断

Epperson 等^[4]通过五名具有多智能体开发经验的实践者访谈，发现长对话检查、交互调试和配置迭代中的困难。参与者需要阅读大量文本消息，理解运行过程并寻找出错位置；模型输出的随机性还使修改效果需要多次运行才能判断。该研究为智能体排错困难提供了直接的开发者经验材料。

该研究后续的错误识别实验中，参与者使用交互调试工具和基础界面产生的错误描述质量相当，在开始编辑消息前仍花费较多时间阅读记录。运行引导研究还发现，参与者对智能体实现的理解影响其修改方式，修改后也可能难以判断新

运行是否产生预期变化。上述结果具体表明，在这些任务中，记录访问和编辑能力已经可用，理解运行过程并形成可检查的原因判断仍然需要投入。其研究对象为特定多智能体团队，正式任务集中于两项网络信息查询任务；工具调用型智能体的其他排错情境还需要结合实际事件考察。

2.2.2 信息查找与系统理解

程序调试研究提供了分析这些困难的基础。Katz 和 Anderson^[13]通过程序调试实验区分位置搜索与局部行为判断。Ko 等^[14]观察十名开发者完成维护任务，将其活动概括为查找、关联和收集相关信息，发现信息线索和导航成本会影响程序理解。Lawrance 等^[15]从信息觅食（information foraging）视角分析程序员调试行为，说明线索、预期价值和访问成本如何影响下一步搜索。

Starr 和 Storey^[16]访谈二十七名专业开发者，将排错经历中的困惑与系统理解、心理模型构建及认知疲劳联系起来。Romero 等^[17]研究多表征软件环境中的调试策略，发现表征使用受到编程经验、表征知识和任务的影响。Parnin 和 Orso^[18]通过自动故障定位工具的用户研究指出，取得可疑代码排序之后，开发者仍需上下文才能理解和修复问题。

这些研究共同说明，调试中的困难涉及相关信息的取得，也涉及如何把信息组织为原因解释。对于工具调用型智能体，提示、调用参数、工具返回和状态记录是可能需要联合检查的材料。上述程序调试研究为观察搜索和理解活动提供参照；哪些材料之间的联系最难判断，以及这种困难是否在已有工具下持续出现，需要由智能体开发者的实际任务确定。

2.2.3 生成式系统中的评价与协作

van der Maden 等^[5]访谈十九名大语言模型产品实践者，发现团队在将评价结果转化为具体改进时存在困难。该研究支持对评价后续行动的关注，其涉及的产品、组织和工作安排较为广泛，具体困难是否来自运行记录的组织方式，还需要任务层面的证据。

Barke 等^[19]观察二十名程序员使用代码生成工具，发现已有明确实现意图和探索陌生任务时的交互方式存在差异。Kumar 等^[6]研究十九名开发者与编程智能体协作处理三十三个真实仓库问题的过程，记录了持续评价、增量交互和人工干预。这些工作揭示了开发者使用生成工具完成任务时的理解与判断活动。搭建或维护智能体的开发者还需排查工具接口、状态更新及运行配置，相关信息需要应在本课题的目标人群中进一步确认。

现有实证研究已经揭示了长轨迹阅读、上下文理解和运行调整等困难。本课题可继续研究的重点，是这些困难在目标排错情境中如何具体阻碍原因判断，以及已有工具的哪些能力仍未充分帮助开发者完成该判断。形成性研究需要记录现用工具、已取得的信息、判断受阻的位置和实际后果，才能为后续设计提供依据。

2.3 现有交互工具的能力与评价

2.3.1 从运行结果进入相关记录

Cornelissen 等^[20]通过受控实验研究执行轨迹可视化对程序理解的影响，在其任务设置下观察到正确性和完成时间的改善。Ko 和 Myers^[21]提出面向原因问答的调试界面（Whyline），支持开发者从可见输出提出问题，再回查相关执行事件，并通过用户任务检验其作用。这些工作说明，运行记录的组织以及从问题进入记录的方式能够影响任务表现。

Lu 等^[22]提出智能体行为可视分析系统（AgentLens），通过层级时间视图和行为原因追踪支持运行探索。其十四名参与者的对照研究测量了任务正确率和完成时间，任务包含个体行为、行为原因及涌现现象分析，报告了相应表现改善。该证据支持社会模拟情境中的行为分析，其任务与开发者从工具调用失败中判断故障原因仍有差异。

Gao 等^[23]提出科学智能体执行轨迹可视化工具（Graph of Trace），将运行事件组织为可交互的图，并通过专家评价了解其使用价值。Wu 等^[24]提出轨迹转图分析平台（AgentGraph），以带类型的节点和关系表示任务、工具、数据及干预，

将图元素链接到原始轨迹片段，并展示失败分析和扰动测试。这些系统已经提供事件关联与出处访问能力。进一步评价其对错误溯源的帮助，需要对应到开发者实际完成的原因判断任务。

上述工具提供了分层浏览、关系导航和原始记录访问等不同实现。其比较需要区分新增记录、自动分析结果和界面组织各自带来的变化。例如，增加数据关系的采集会改变可见信息，调整已有记录的排列则改变访问方式，两类方案能够支持的效果解释不同。

2.3.2 比较与结构化复盘

Khanna 等^[25]通过人工智能复盘工具（AAR/AI）研究结构化复盘对故障发现的作用。其对照实验涉及六十五名具有即时战略游戏经验、没有人工智能或机器学习背景的参与者；两种条件均可查看游戏信息及系统解释，复盘条件进一步组织对预期、实际行为和原因的检查。研究对故障定位与描述评分，报告了查准率和召回率的改善。这为组织检查过程能够改善故障判断提供了实证依据。

Arawjo 等^[26]提出提示与输出比较工具（ChainForge），支持提示、模型和输入的组合试验，并通过实验室研究与访谈考察输出探索、假设检验和评价迭代。它提供了比较多个输出的操作空间。工具调用型智能体还涉及中间输出如何被后续步骤使用，比较哪些内容能够帮助解释一次任务失败，需要结合具体溯源困难确定。

结构化复盘和输出比较已经具有研究基础。应用于智能体错误溯源时，需要明确开发者正在区分什么原因、比较哪些记录，以及比较结果能支持什么结论。这些具体条件决定设计能否减少无关查找或帮助识别关键差异。

2.3.3 逐步检查与运行干预

Dibia 等^[27]提出多智能体开发工具（AutoGen Studio），整合组件配置、执行查看和调试功能。Rorseth 等^[28]提出面向数据应用的智能体调试器（LADYBUG），展示逐步追踪、局部干预和受影响环节重新执行。两项系统工作提供了可实现的

调试能力。

Epperson 等^[4]据开发者访谈设计交互调试工具（AGDebugger），提供消息浏览、编辑、重置和对话概览。六人的错误识别研究及八人的运行引导研究表明，参与者重视编辑和重执行操作，同时还面临前述运行理解与修改效果判断困难。研究为继续改进这些具体活动提供了依据。

Hutter 和 Pradel^[29]提出软件开发智能体调试器（AgentStepper），支持结构化轨迹、逐步检查、编辑和中间代码变更查看。其十二名参与者的对照研究包含一项轨迹理解和两项缺陷定位任务，参与者为计算机专业学生和博士生。研究报告了缺陷定位成功率的改善；轨迹理解任务的表现差异较小。研究还指出，中间代码变更记录能够揭示工具返回成功与实际变更之间的不一致，事件摘要则帮助参与者检索信息。因此，结果支持该组合工具在所测任务中的作用，其中记录能力与呈现方式均可能影响定位表现。

现有研究已经把“发现困难、提出工具、评价效果”用于智能体调试。继续开展交互设计研究，需要从已有能力下仍存在的困难出发，说明具体改变哪项信息或操作，以及该改变如何帮助开发者判断失败原因。

2.3.4 工具研究的综合比较

表 1.1 比较相关研究的任务、能力与结果。最后一列依据各研究的任务范围和评价方式归纳证据边界，作为确定本课题研究范围的依据。

表 1.1 错误溯源相关工具的能力与证据范围

研究与任务	主要能力	评价及结果	证据边界
Whyline ^[21] ：程序输出原因查询	从输出问题进入相关执行事件	用户实验检验问题入口与执行事件连接的作用	面向具有明确程序语义的执行记录

表 1.1 错误溯源相关工具的能力与证据范围（续）

研究与任务	主要能力	评价及结果	证据边界
AAR/AI ^[25] : 游戏人工智能故障发现	组织预期、实际行为与原因的复盘	65 人对照实验，故障发现的查准率和召回率改善	面向领域用户，任务为游戏中的故障识别与描述
ChainForge ^[26] : 提示与输出检验	组合试验、输出比较	实验室研究和访谈揭示探索与假设检验方式	评价围绕提示及模型输出，跨步骤失败追查需另行考察
AgentLens ^[22] : 模拟智能体行为分析	层级轨迹、行为原因追踪	14 人对照研究，测量正确率与时间并报告改善	主要任务为社会模拟中的行为及原因分析
AGDebugger ^[4] : 多智能体错误识别与引导	浏览、编辑、重置与对话概览	诊断部分两条件错误描述质量相当；参与者偏好交互操作	两项网络查询任务中仍有运行理解与修改效果判断困难
AgentStepper ^[29] : 编程智能体缺陷定位	结构化轨迹、中间代码变更与逐步检查	12 人对照研究，报告缺陷定位改善	学生及博士生样本；整体效果包含记录与呈现等多项能力
Graph of Trace ^[23] : 科学工作流程检查	细粒度事件记录与交互事件图	专家评价提供使用价值方面的证据	对开发者错误溯源表现需有相应任务评价
AgentGraph ^[24] : 轨迹分析与鲁棒性测试	类型化关系、原始出处访问与扰动测试	系统演示展示失败分析和测试能力	系统能力的展示尚需与具体用户溯源任务相对应

不同研究已经分别提供行为分析、故障发现和缺陷定位的效果证据。它们采用的任务、可见信息和功能组合不同，结果需要在各自条件下理解。对本课题而言，比较应围绕形成性研究确认的一项困难展开，并明确两种方案之间究竟改变了什么。

2.3.5 评价中的判断与使用体验

Pareek 等^[30]让十二名参与者使用五种多智能体界面完成任务，分别收集透明性、帮助程度和可靠性判断。Kaur 等^[31]研究数据科学家使用解释工具的情况，观察到误解和过度依赖。Sieker 等^[32]在视觉问答任务中发现，提供解释提高了使用者对系统能力的评价，却未帮助其更准确地识别系统限制。这些研究说明，使用者的主观理解需要与实际判断表现分别考察。

Buçinca 等^[33]发现，促使使用者独立思考的交互措施能够减少对错误建议的过度依赖，同时伴随接受度和使用成本变化。由这些相邻任务的结果可提出评价要求：错误溯源工具的效果应同时结合原因判断及其依据、完成时间和使用体验解释。它们为测量选择提供参照，目标情境中是否存在相同偏差仍需实际观察。

2.4 研究不足与本课题切入点

2.4.1 已有研究形成的基础

现有文献已经提供三方面基础。首先，开发者访谈与任务研究揭示了长轨迹检查、上下文理解和运行调整中的困难。其次，运行追踪、自动归因以及交互工具提供了事件查找、关系展示、输出比较和运行干预等能力。再次，部分工具已经通过对照实验报告了行为分析或缺陷定位方面的改善。上述成果共同支持继续研究交互设计如何帮助开发者查明智能体失败原因。

2.4.2 尚需明确的具体问题

第一，已有能力下仍然阻碍原因判断的具体困难，需要在目标排错情境中明确。Epperson 等^[4]记录了大量阅读、实现理解和修改效果判断方面的困难；Hutter 和 Pradel^[29]的研究则显示，工具返回信息与实际状态变化的对照能够帮助定位特定缺陷。这些发现提示，开发者取得相关记录后，还需要判断哪些记录能够解释失败。现有案例分别涉及多智能体网络查询和编程智能体，尚不能直接据此确定本课题的工具调用场景中应优先改善哪项活动。需要结合实际排错事件，识别受

阻的判断、当时使用的工具与材料，以及由此产生的检查或修改后果。

第二，针对这些困难的具体设计选择，还需要与对应的任务表现联系起来。AgentLens 和 AgentStepper 的研究已经报告整体工具的作用，但其方案同时包含记录、摘要、关系追踪或视图组织等多项能力。后续设计需要说明某项操作针对什么困难、与已有方式存在什么可比较差异，并通过相应任务检验其作用。对于错误溯源，原因判断是否符合运行事实、所引用记录是否支持该判断，是判断设计价值的重要依据。

因此，本课题关注开发者从可疑步骤形成有依据的失败原因判断这一过程。该过程直接影响后续检查和修改的位置选择，也为信息查找、上下文关联及原因比较提供具体设计任务。形成性研究将区分关键事实未被记录、已有材料难以访问或关联，以及材料已经取得但原因仍难以判断三类情况，选择其中有实际影响、且能够通过交互设计改善的困难。

2.4.3 研究定位

本研究面向工具调用型智能体开发者，按“发现问题—提出设计方案—验证方案”开展。首先考察开发者使用现有工具进行错误溯源的困难，随后围绕其中一项关键困难提出并迭代交互设计，最后与相应的基础工具比较原因判断表现和完成成本。具体设计功能由前期研究确定。

研究预期形成目标情境中的溯源困难描述、针对关键困难的设计方案及效果证据。对判断依据的评分和新案例检验用于提高评价质量、明确结论适用范围，并与所选困难及设计内容保持一致。

3 研究问题与方法

3.1 研究问题

本研究面向工具调用型大语言模型智能体的开发者，关注任务失败后利用运行记录查明失败原因的过程。围绕“发现问题—提出设计方案—验证方案”提出三个研究问题：

研究问题一（RQ1）：开发者使用现有工具进行智能体错误溯源时，存在哪些困难？

研究问题二（RQ2）：如何通过交互设计缓解其中的关键困难？

研究问题三（RQ3）：与现有工具相比，所提出的交互设计能否改善开发者的错误溯源表现？

RQ1 通过访谈和任务观察回答，结果用于选择 RQ2 所针对的关键困难；RQ2 通过方案比较和原型迭代形成设计；RQ3 通过对照任务评价其效果。错误溯源表现主要指原因判断及其依据的质量，完成时间和使用体验用于补充解释结果。

3.2 研究对象

研究对象为具有实际开发或排错经验的工具调用型智能体开发者。目标系统包含一个主要智能体，能够通过多次模型请求和工具调用完成任务，并保留可供检查的运行记录。研究任务从失败结果出发，要求开发者判断原因并指出支持判断的运行记录。实际修复系统不作为主要任务要求。

主要分析单位为一次具有明确任务目标、失败表现和诊断活动的事件。案例将覆盖跨步骤信息传递、工具交互及流程控制等运行条件。多智能体研究为相关工作和案例设计提供参考，正式研究范围集中在单个主要智能体，以保持责任关系和任务条件的可分析性。

研究关注对运行行为的诊断，包括工具参数、返回内容、状态更新和配置条件。模型内部表征、训练机制和参数级解释不构成本课题的研究对象。

3.3 理论基础

本研究以任务活动和运行材料为分析起点，将任务症状、可见材料、参与者问题、原因假设、检查行动和判断变化作为初始分析概念，并在开放编码过程中补充和修订。

Lawrance 等^[15]的信息觅食视角用于解释材料入口、搜索线索和访问成本。意义建构（sensemaking）视角用于分析分散材料如何被组织为原因解释，以及解释怎样随新材料改变。Pirolli 和 Card^[34]的认知任务分析提供了信息搜集与解释形成的过程参照；Klein 等^[35]关于资料与解释框架相互作用的讨论，可用于识别假设维持与修订的条件。

资料分析分别编码材料访问、搜索线索、原因假设、判断修订和检查行动。研究保留不能由初始理论框架解释的资料，并据此调整分类。

3.4 研究方法

3.4.1 形成性研究

参与者招募。

形成性研究拟招募 12—16 名参与者，通过公司和领英（LinkedIn）等外部专业渠道招募，覆盖不同组织、开发经验和工具使用背景。纳入条件为：近六个月内实际搭建、修改或维护过工具调用型智能体，且能够描述至少一次亲自参与的排错事件。招募将记录开发年限、智能体经验、系统类型、排错频率和来源渠道，并针对已有样本的缺项补充参与者。

Malterud 等^[36]提出的信息力（information power）将定性样本充分性与目标、样本特征、访谈质量和分析方式联系起来。本研究据此评估资料是否足以回答研究问题，重点检查核心困难是否有跨参与者和跨任务的支持，反例是否得到解释，以及不同经验背景的材料是否充分。计划人数用于安排研究资源，实际招募根据资料质量和案例覆盖进行调整。

资料收集。

每次研究约 60—90 分钟，包括关键事件回溯和错误溯源任务观察。关键事件回溯围绕参与者亲历的故障展开，依次询问任务目标、最初症状、当时使用的工具、已取得的材料、曾怀疑的原因、采取的检查 and 最终判断，并记录判断受阻的位置及其对后续检查或修改的影响。优先请参与者借助获得授权并脱敏的记录说明过程；无法提供记录时，以事件时间线和具体操作回忆补充，并标记资料来源。

任务观察使用初步构建的失败案例，让参与者借助现用或熟悉的工具查阅运行材料并完成原因判断，记录工具能力及其使用方式。无法使用原有工具时，使用具备相应基础能力的界面，并标记这一任务条件。研究记录屏幕、操作和同步口语报告（think-aloud），在自然停顿处询问当前判断及其依据。任务结束后回看关键片段，核对研究者对操作含义的理解。真实事件用于理解工作情境，统一任务便于比较不同参与者在相近条件下的活动。

资料包括访谈转录、事件概要、运行片段、操作时间线、参与者提出的假设及判断变化。研究将区分参与者回忆的经历、任务中直接观察的行为和研究者的解释。

资料分析。

研究采用框架法（framework method）。Gale 等^[37]提出的案例—主题矩阵适合组织多名参与者的过程资料。分析先选取不同背景的案例开放编码，再结合初步分析框架形成编码表，随后在其余资料中应用和修订。

编码内容主要包括：当前溯源问题、使用的工具、所需与实际取得的信息、原因解释、检查行动、判断变化及活动结果。分析区分事实未记录、材料难以访问或关联、已有材料难以解释等情况，并记录困难的持续时间及后续影响。跨案例比较结合口语报告、操作记录和阶段性判断识别重复出现的困难，同时保留其他类型的问题。

研究拟邀请另一名研究人员复核约四分之一的资料，比较编码依据并记录分歧处理。分析保留反例和条件差异，形成核心困难、支持案例及适用条件的矩

阵。结果进入设计阶段前，将邀请部分参与者核对事件解释是否符合其经历。

3.4.2 案例构建

案例来源。

研究以开源智能体框架自行构建可复现案例，并依据实践研究修订其任务和故障条件。初步考虑资料检索、结构化数据查询和多步骤工具处理等任务，最终选择取决于访谈结果及实现可行性。公司案例主要用于理解实际困难和核查任务真实性，正式实验材料以可公开或可授权复现的系统为主。

拟建立 12—16 个候选案例，经过专家核查和预实验后选取 12 个进入正式案例池。每个案例保存任务说明、系统版本、工具定义、正常运行材料、失败运行材料、配置及故障构造记录。保留完成诊断所需的背景和合理干扰事件，控制内容长度与领域知识要求。

案例条件依据形成性研究中的高频困难和差异条件确定。正式实验选择一项与核心设计主张直接相关的案例条件作为主要比较因素，其余条件用于平衡任务难度。

证据评定。

每个案例建立两份记录。故障记录说明实际改变了哪些系统条件、故障如何发生，以及哪些复现或干预支持这一解释。任务记录说明参与者可看到哪些材料、能够据此作出什么判断，以及仍有哪些无法确定的内容。

正式比较优先使用材料能够支持原因判断的案例，要求参与者指出原因及相关运行记录。若形成性研究确认信息缺失直接关系到所选困难，可另设补充案例，观察参与者能否指出具体缺口和合理的后续检查。两类任务分别评分和解释，主要效果比较使用前一类案例。

案例拟由两名具有相关开发经验的人员独立检查。核查内容包括情境合理性、故障可复现性、可见材料充分性、替代解释和评分规则。存在多个有效诊断路径时，将其纳入评分手册。构造时选定的故障只是实际原因记录，评分还需依

据参与者可见材料判断答案是否合理。

3.4.3 原型设计

本阶段采用通过设计开展研究（research through design）的方法。Zimmerman 等^[38]讨论了设计实践与研究贡献的联系，Sedlmair 等^[39]及 Meyer 和 Dykes^[40]强调问题理解、设计取舍和验证之间的对应。本研究据此记录每种方案针对的困难、预期作用、使用证据及修改理由。

研究围绕形成性研究确认的一项关键困难提出至少两种备选方案。例如，若困难集中于查找某次工具返回如何被后续步骤使用，可比较关联导航与并列查看；若困难集中于区分相似原因，可比较不同的材料比较方式。具体方案依据观察结果确定，并与已有工具的相应能力对照。

设计探索拟开展两轮任务式走查，共招募 6—8 名参与者，每轮约 4—5 人，部分参与者参与两轮以观察修改效果，并补充首次使用者检查学习成本。走查使用相同或难度相近的任务，收集理解错误、检查行为、未满足的信息需要和操作障碍。

原型分为通用功能和研究功能。通用功能包括任务说明、事件查看、必要搜索和材料展开；研究功能承载核心设计主张。每次迭代保存对应困难、设计意图、观察结果及修改理由，淘汰仅受偏好支持但未改善目标活动的方案。

最终原型优先使用保存的运行材料，以保证任务可复现。若选定困难需要通过重执行检查原因解释，将同步提供受控执行、修改记录和结果比较，并相应调整任务与评分。正式评价前冻结原型版本。

3.4.4 对照实验

实验条件。

正式研究采用被试内设计（within-subject design），比较基础界面与研究界面。基础界面依据形成性研究中实际使用的工具确定，提供时间序列、事件详情、搜索和展开等操作；现有工具中与目标困难相关的能力也纳入比较条件。研

究界面在此基础上实现选定设计。能够直接用于任务的现有工具优先作为基础条件；需要复现其能力时，记录复现范围。训练使用独立练习案例，确保参与者能够操作两种界面。

比较前建立条件清单，逐项核对原始材料、派生内容、搜索能力、默认展开状态和访问步骤。若研究重点为信息组织，两种界面提供相同语义内容，差异集中于其呈现和操作。若选定方案引入新的派生信息，则把“派生信息及其呈现”的整体作用作为评价对象，并在研究主张和分析中保持一致。

正式评价比较两种界面下有依据的原因判断达成率，并考察完成时间和使用负担。结论对应实际采用的比较工具、功能和案例范围。形成性研究确定具体设计后提出效果假设，预实验完成后预注册假设、主要指标和评分规则。

任务分配。

正式实验的招募条件与形成性研究一致，并记录参与者经验。预实验拟招募4—6人，用于检查任务时长、理解难度、评分可操作性和测量流程。预实验参与者不进入正式实验；正式实验参与者不提前接触所用案例。

正式案例池初步按12个案例规划，每人完成6个，每种界面各3个。最终案例数量和单人任务量依据预实验中的任务时长和难度确定。通过平衡任务分配，使每个案例在两种界面下均获得评价，同一参与者只接触每个案例一次。界面顺序与案例分配交叉平衡，避免将某类故障固定给某一条件。全部任务时长和休息安排依据预实验确定。

样本量依据主要指标、最小有实际意义的差异、参与者和案例的变异范围进行模拟论证。Lakens^[41]的样本量论证方法以及Green和MacLeod^[42]的模拟方法为此提供依据。研究在正式招募前确定目标人数和分析计划，将预实验参数与保守参数范围共同纳入计算，避免仅依赖单一试测效应。

评价指标。

每个主要任务要求参与者提交失败原因判断及支持判断的运行事件。正式计时任务不要求持续口述，过程资料以操作记录为主，并在任务后进行简短回

顾，了解查找困难及判断依据。信息不足的补充案例另外收集材料缺口与后续检查，不计入主要任务的达成率。

主要指标为有依据的原因判断达成率，即同时满足以下两个条件的任务占全部主要任务的比例：

- 1) 原因判断正确，并说明相关步骤与任务失败的联系。
- 2) 引用的运行事件准确且能够支持关键判断。

两个条件分别评分，并保留原因判断与依据使用的分项结果。评分手册根据案例允许的合理解释和诊断路径制定，在预实验中检查可操作性。完成时间作为效率指标，操作记录和任务后回顾用于解释效果出现的环节。

表 1.2 诊断评价指标

指标类别	具体测量	解释用途
主要结果	有依据的原因判断达成率	比较两种界面下的错误溯源表现
判断组成	原因判断、依据使用的分项得分	区分判断正确性与依据使用情况
时间与操作	完成时间、关键材料访问、比较与回查行为	描述获得结果所需的成本及过程
使用体验	心智努力、操作困难及访谈反馈	解释使用负担和接受程度

任务达到时间上限时结束并保存当前判断；完成时间按未完成任务的截尾情况处理。两名评分者依据匿名化答案独立评分，评分材料隐藏界面条件，先报告一致性，再按评分手册讨论分歧。评分手册在正式数据分析前冻结。

数据分析。

主要结果拟使用广义线性混合模型（generalized linear mixed model），以界面条件为主要固定效应，纳入参与者与案例的随机效应，并按预注册计划处理顺序

和经验因素。模型结构、参数估计及拟合异常的处理方式在预实验后确定。结果报告效应估计及置信区间。

时间、分项得分和使用体验分别分析。过程资料用于检查关键材料是否被发现、原因解释是否得到核验，以及错误判断如何形成。经验和案例条件的差异主要用于描述效果范围，探索性分析与主要检验分别呈现。

研究将主要结果、分项得分和过程资料与设计的预期作用对应分析。结论范围依据实际出现的诊断表现、检查行为和使用成本确定。

3.4.5 迁移检验

对照评价后，拟以 3—4 个未参与设计迭代的新案例开展补充走查，改变任务内容或工具组合，保留相同的核心溯源困难。拟招募 6—8 名具有相关经验的参与者，观察设计在这些条件下的使用过程和失效情境。该阶段用于说明效果适用范围，其实施规模依据主要评价结果及待解释的问题确定。

设计建议结合观察、原型迭代和评价结果整理。Amershi 等^[43]及 Weisz 等^[44]的工作提供了通过多种使用情境形成和修订设计原则的方法参照。建议说明适用困难、设计操作、支持证据与实施条件，推广范围依据实际覆盖的案例确定。

每项原则按照以下结构整理：

表 1.3 设计原则的表述结构

内容	需要回答的问题
适用情境	哪类任务、信息条件或使用者困难需要这项原则？
设计操作	设计者应怎样组织信息或提供检查操作？
预期作用	该操作影响哪项诊断活动？
支持证据	哪些观察、比较结果和新案例支持这一判断？
实施条件与成本	需要哪些记录、背景知识和操作投入？
适用边界	哪些情况下效果减弱、无效或产生新的困难？

研究保留未得到支持的设计主张，并说明其结果。最终原则数量根据证据决定。

3.4.6 质量控制

本研究重点是建立实践问题、设计选择与评价结论之间的对应关系。形成性研究确定可重复观察的困难，原型表达针对这一困难的设计主张，实验检查主张是否获得支持，新案例研究界定适用条件。

另一个重点是诊断答案的合理性。研究将区分实际故障、可见证据和参与者判断，避免只按最终原因名称评价任务表现。原始材料、派生信息和自动建议保留来源，使评分和过程分析具有可追溯依据。

表 1.4 研究风险控制

难点	应对安排
实际案例包含公司或个人信息	以授权脱敏事件理解情境，用开源系统构建可复现任务
自建案例与真实困难脱节	依据访谈选择案例，由实践者核查背景、歧义和检查路径
多种原因符合表面症状	分别记录实际原因与可见材料支持的解释，允许多条合理诊断路径
原型同时改变过多因素	围绕一个核心问题设计条件，核对各条件的信息和操作差异
参与者经验与案例难度造成差异	记录经验，训练基础操作，平衡任务，联合分析人员与案例差异
专业参与者招募不足	提前开展公司与外部渠道招募，分阶段维护招募矩阵并调整计划
模型和工具更新影响重复评价	固定运行材料、版本与配置，保存构造记录及可复现环境

3.5 可行性分析

课题来自实际开发工作中的问题观察，已完成文献调研和方案设计。访谈、案例、原型和实验尚未开展，文中样本及研究安排均属计划。

研究者可通过公司渠道接触若干具有智能体开发经验的人员，并计划通过外部专业社群扩展招募。技术实现拟使用开源框架，研究原型主要处理保存的运行材料和必要的检查操作，开发范围与核心研究问题保持一致。

整体工作按阶段推进。先以访谈和轻量案例确认问题，再通过低保真方案和任务走查确定设计，正式实现集中于评价需要的功能。中期前完成形成性研究、原型和预实验，并争取取得正式评价的阶段性数据；中期后完成评价、迁移检验和论文写作。

3.6 研究伦理

涉及参与者的研究将在完成学校要求的伦理程序后开展。知情同意材料说明研究目的、参与内容、录音及屏幕记录范围、补偿方式、退出权利和数据保存安排。公司渠道招募遵循自愿原则，参与情况不用于工作评价。

研究仅收集完成分析所需的材料。真实案例须取得授权并进行脱敏，排除密钥、个人信息及不可公开的业务内容。联系信息与研究编号分开保存，原始录音、屏幕记录和文本资料限制访问。论文使用匿名编号和必要摘录，公开材料遵守参与者授权及开源许可。

自动生成的摘要或建议保留生成配置与来源标记。案例、评分手册、原型版本和分析材料统一归档，以支持研究复核。

4 研究框架

本研究以“发现问题—提出设计方案—验证方案”为主线。形成性研究识别现有工具下的溯源困难，设计探索围绕一项关键困难形成原型，对照评价检验原因判断表现。新案例走查及设计建议整理作为验证阶段的补充。各阶段产出与

三个研究问题对应。

表 1.5 研究实施阶段

阶段	核心工作	阶段产出	进入下一阶段的依据
诊断实践研究	关键事件访谈、任务观察、跨案例分析	困难与信息条件矩阵	核心困难有多个案例支持，能够通过设计进行比较
设计探索	提出备选方案、任务式走查、迭代原型	明确设计主张与原型	方案可用，设计差异可描述，评价材料能够回答问题
对照评价	冻结条件、开展正式任务、分析判断和过程	效果估计与行为解释	获得足以讨论核心主张的任务及过程资料
新案例检验	更换任务或工具条件，观察作用变化	支持案例、失效案例及修订依据	能够说明原则适用与不适用的条件
知识提炼	综合实证材料与设计记录	经验描述和设计原则	原则各项论断均有对应材料与证据范围

核心困难的选择依据包括出现范围、诊断后果、材料可获得性、设计可介入性和实验可检验性。原型围绕最终选定的问题实现必要功能。

5 计划进度

表 1.6 研究计划

时间	研究工作	阶段成果
2026.09— 2026.10	完成开题，细化研究协议，办理伦理程序，构建初始案例	研究协议、招募材料、案例模板

表 1.6 研究计划（续）

时间	研究工作	阶段成果
2026.11— 2027.01	开展关键事件访谈与任务观察，同步整理资料	访谈及观察资料、初步编码
2027.02— 2027.03	跨案例分析，补充样本，确定核心设计问题	困难与信息条件矩阵、设计要求
2027.03— 2027.05	比较备选方案，完成两轮任务式走查	设计记录、核心主张、原型方案
2027.05— 2027.06	实现研究原型，核查案例与评分规则	原型、正式案例池、评分手册
2027.06— 2027.08	预实验、样本量论证与预注册，启动正式评价	预实验结果、正式研究计划、阶段数据
2027.09	中期答辩	形成性研究、设计依据、原型及评价进展
2027.09— 2027.10	完成正式评价与新案例检验	完整研究数据、原则修订依据
2027.11— 2028.02	综合分析并撰写论文	经验描述、设计原则、论文初稿
2028.03— 2028.05	论文修改、送审与答辩准备	学位论文定稿、归档材料

参考文献

- [1] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C/OL]//International Conference on Learning Representations. 2023. arXiv: 2210.03629. <https://arxiv.org/abs/2210.03629>.
- [2] Cemri M, Pan M Z, Yang S, et al. Why Do Multi-Agent LLM Systems Fail? [C/OL]//Advances in Neural Information Processing Systems: vol. 38. 2025. https://papers.nips.cc/paper_files/paper/2025/hash/b1041e52d3be19f0a9bc491657488e4a-Abstract-Datasets_and_Benchmarks_Track.html. DOI: 10.52202/085713-4082.
- [3] Chen M, Wang J, Liu Z, et al. From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws[PP/OL]. arxiv(2026) [2026-09-20]. <https://arxiv.org/abs/2606.06324v2>. arXiv: 2606.06324 [cs.SE].
- [4] Epperson W, Bansal G, Dibia V, et al. Interactive Debugging and Steering of Multi-Agent AI Systems[C/OL]//Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems. 2025: 156:1-156:15. DOI: 10.1145/3706598.3713581.
- [5] Van der Maden W, Sadek M, Xiao Z, et al. Results-Actionability Gap: Understanding How Practitioners Evaluate LLM Products in the Wild[C/OL]//Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems. 2026: 1-17. DOI: 10.1145/3772318.3791069.
- [6] Kumar A, Bajpai Y, Gulwani S, et al. Why AI Agents Still Need You: Findings from Developer-Agent Collaborations in the Wild[C/OL]//Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering.

- 2025: 432-444. <https://arxiv.org/abs/2506.12347v3>. DOI: 10.1109/ASE63991.2025.00043.
- [7] Höök K, Löwgren J. Strong Concepts: Intermediate-Level Knowledge in Interaction Design Research[J/OL]. ACM Transactions on Computer-Human Interaction, 2012, 19(3): 23:1-23:18. DOI: 10.1145/2362364.2362371.
 - [8] Fonseca R, Porter G, Katz R H, et al. X-Trace: A Pervasive Network Tracing Framework[C/OL]//Proceedings of the 4th USENIX Symposium on Networked Systems Design and Implementation. 2007: 271-284. <https://www.usenix.org/conference/nsdi-07/x-trace-pervasive-network-tracing-framework>.
 - [9] Sigelman B H, Barroso L A, Burrows M, et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure: dapper-2010-1[R/OL]. Google. 2010. <https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/>.
 - [10] Mace J, Roelke R, Fonseca R. Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems[C/OL]//Proceedings of the 25th ACM Symposium on Operating Systems Principles. 2015: 378-393. DOI: 10.1145/2815400.2815415.
 - [11] Zheng Y, Hu Y, Yu T, et al. AgentSight: System-Level Observability for AI Agents Using eBPF[PP/OL]. arxiv(2025) [2026-09-20]. <https://arxiv.org/abs/2508.02736v2>. arXiv: 2508.02736.
 - [12] Zhang S, Yin M, Zhang J, et al. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems[C/OL]//Proceedings of Machine Learning Research: vol. 267. 2025: 76583-76599. <https://proceedings.mlr.press/v267/zhang25cq.html>.
 - [13] Katz I R, Anderson J R. Debugging: An Analysis of Bug-Location Strategies [J/OL]. Human-Computer Interaction, 1987-1988, 3(4): 351-399. DOI: 10.1207

/s15327051hci0304_2.

- [14] Ko A J, Myers B A, Coblenz M J, et al. An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks[J/OL]. IEEE Transactions on Software Engineering, 2006, 32(12): 971-987. DOI: 10.1109/TSE.2006.116.
- [15] Lawrance J, Bogart C, Burnett M, et al. How Programmers Debug, Revisited: An Information Foraging Theory Perspective[J/OL]. IEEE Transactions on Software Engineering, 2013, 39(2): 197-215. DOI: 10.1109/TSE.2010.111.
- [16] Starr A, Storey M-A. Theory of Troubleshooting: The Developer's Cognitive Experience of Overcoming Confusion[J/OL]. ACM Transactions on Software Engineering and Methodology, 2026. DOI: 10.1145/3800945.
- [17] Romero P, du Boulay B, Cox R, et al. Debugging Strategies and Tactics in a Multi-Representation Software Environment[J/OL]. International Journal of Human-Computer Studies, 2007, 65(12): 992-1009. DOI: 10.1016/j.ijhcs.2007.07.005.
- [18] Parnin C, Orso A. Are Automated Debugging Techniques Actually Helping Programmers?[C/OL]//Proceedings of the 20th International Symposium on Software Testing and Analysis. 2011: 199-209. DOI: 10.1145/2001420.2001445.
- [19] Barke S, James M B, Polikarpova N. Grounded Copilot: How Programmers Interact with Code-Generating Models[J/OL]. Proceedings of the ACM on Programming Languages, 2023, 7(OOPSLA1): 78:1-78:27. DOI: 10.1145/3586030.
- [20] Cornelissen B, Zaidman A, van Deursen A. A Controlled Experiment for Program Comprehension through Trace Visualization[J/OL]. IEEE Transactions on Software Engineering, 2011, 37(3): 341-355. DOI: 10.1109/TSE.2010.47.

- [21] Ko A J, Myers B A. Finding Causes of Program Output with the Java Whyline [C/OL]//Proceedings of the 27th ACM Conference on Human Factors in Computing Systems. 2009: 1569-1578. DOI: 10.1145/1518701.1518942.
- [22] Lu J, Pan B, Chen J, et al. AgentLens: Visual Analysis for Agent Behaviors in LLM-Based Autonomous Systems[J/OL]. IEEE Transactions on Visualization and Computer Graphics, 2025, 31(8): 4182-4197. DOI: 10.1109/TVCG.2024.3394053.
- [23] Gao T, Li H, Li J H, et al. Graph of Trace: Visualizing Execution Traces of Scientific Agents[C/OL]//Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics: System Demonstrations. 2026: 297-306. <https://aclanthology.org/2026.acl-demo.29/>.
- [24] Wu Z, Cho S, Villalobos C E M, et al. AgentGraph: Trace-to-Graph Platform for Interactive Analysis and Robustness Testing in Agentic AI Systems[C/OL]//Proceedings of the AAAI Conference on Artificial Intelligence: vol. 40: 48. 2026: 41721-41723. DOI: 10.1609/aaai.v40i48.42393.
- [25] Khanna R, Dodge J, Anderson A, et al. Finding AI's Faults with AAR/AI: An Empirical Study[J/OL]. ACM Transactions on Interactive Intelligent Systems, 2022, 12(1): 1:1-1:33. DOI: 10.1145/3487065.
- [26] Arawjo I, Swoopes C, Vaithilingam P, et al. ChainForge: A Visual Toolkit for Prompt Engineering and LLM Hypothesis Testing[C/OL]//Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems. 2024. DOI: 10.1145/3613904.3642016.
- [27] Dibia V, Chen J, Bansal G, et al. AutoGen Studio: A No-Code Developer Tool for Building and Debugging Multi-Agent Systems[C/OL]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System

- Demonstrations. 2024: 72-79. DOI: 10.18653/v1/2024.emnlp-demo.8.
- [28] Rorseth J, Godfrey P, Golab L, et al. LADYBUG: An LLM Agent DeBUGger for Data-Driven Applications[C/OL]//Proceedings of the 28th International Conference on Extending Database Technology. 2025: 1082-1085. DOI: 10.48786/EDBT.2025.94.
 - [29] Hutter R, Pradel M. AgentStepper: Interactive Debugging of Software Development Agents[PP/OL]. arxiv(2026) [2026-09-20]. <https://arxiv.org/abs/2602.06593>. 3. arXiv: 2602.06593.
 - [30] Pareek S, Govers J, Kollerup N K, et al. Sensemaking in Multi-Agent LLM Interfaces: How Users Interpret Transparency and Trustworthiness Cues[C/OL]//Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems. 2026: 1-20. DOI: 10.1145/3772318.3791157.
 - [31] Kaur H, Nori H, Jenkins S, et al. Interpreting Interpretability: Understanding Data Scientists' Use of Interpretability Tools for Machine Learning[C/OL]//Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems. 2020: 1-14. DOI: 10.1145/3313831.3376219.
 - [32] Sieker J, Junker S, Utescher R, et al. The Illusion of Competence: Evaluating the Effect of Explanations on Users' Mental Models of Visual Question Answering Systems[C/OL]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. 2024: 19459-19475. DOI: 10.18653/v1/2024.emnlp-main.1084.
 - [33] BUÇINCA Z, Malaya M B, Gajos K Z. To Trust or to Think: Cognitive Forcing Functions Can Reduce Overreliance on AI in AI-Assisted Decision-Making[J/OL]. Proceedings of the ACM on Human-Computer Interaction, 2021, 5(CSCW1): 188:1-188:21. DOI: 10.1145/3449287.

- [34] Pirolli P, Card S K. The Sensemaking Process and Leverage Points for Analyst Technology as Identified Through Cognitive Task Analysis[C/OL]//Proceedings of the 2005 International Conference on Intelligence Analysis. 2005. https://www.e-education.psu.edu/geog885/sites/www.e-education.psu.edu/geog885/files/geog885q/file/Lesson_02/Sensemaking_206_Camera_Ready_Paper.pdf.
- [35] Klein G, Moon B, Hoffman R R. Making Sense of Sensemaking 2: A Macro-cognitive Model[J/OL]. IEEE Intelligent Systems, 2006, 21(5): 88-92. DOI: 10.1109/MIS.2006.100.
- [36] Malterud K, Siersma V D, Guassora A D. Sample Size in Qualitative Interview Studies: Guided by Information Power[J/OL]. Qualitative Health Research, 2016, 26(13): 1753-1760. DOI: 10.1177/1049732315617444.
- [37] Gale N K, Heath G, Cameron E, et al. Using the Framework Method for the Analysis of Qualitative Data in Multi-Disciplinary Health Research[J/OL]. BMC Medical Research Methodology, 2013, 13: 117. DOI: 10.1186/1471-2288-13-117.
- [38] Zimmerman J, Forlizzi J, Evenson S. Research through Design as a Method for Interaction Design Research in HCI[C/OL]//Proceedings of the SIGCHI Conference on Human Factors in Computing Systems. 2007: 493-502. DOI: 10.1145/1240624.1240704.
- [39] Sedlmair M, Meyer M, Munzner T. Design Study Methodology: Reflections from the Trenches and the Stacks[J/OL]. IEEE Transactions on Visualization and Computer Graphics, 2012, 18(12): 2431-2440. DOI: 10.1109/TVCG.2012.213.
- [40] Meyer M, Dykes J. Criteria for Rigor in Visualization Design Study[J/OL]. IEEE Transactions on Visualization and Computer Graphics, 2020, 26(1): 87-97. DOI: 10.1109/TVCG.2019.2934539.

- [41] Lakens D. Sample Size Justification[J/OL]. Collabra: Psychology, 2022, 8(1): 33267. DOI: 10.1525/collabra.33267.
- [42] Green P, MacLeod C J. SIMR: An R Package for Power Analysis of Generalized Linear Mixed Models by Simulation[J/OL]. Methods in Ecology and Evolution, 2016, 7(4): 493-498. DOI: 10.1111/2041-210X.12504.
- [43] Amershi S, Weld D, Vorvoreanu M, et al. Guidelines for Human-AI Interaction [C/OL]//Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. 2019: 1-13. DOI: 10.1145/3290605.3300233.
- [44] Weisz J D, He J, Muller M, et al. Design Principles for Generative AI Applications [C/OL]//Proceedings of the CHI Conference on Human Factors in Computing Systems. 2024. DOI: 10.1145/3613904.3642466.

华中科技大学

研究生（文献阅读与选题）报告

☐文献阅读、☒选题、☐文献阅读与选题、☐论文中期进展

标题：工具调用型大语言模型智能体故障诊断的信息表征与交互设计研究

学号 M202570493

姓名 陈思静

专业 设计学

指导教师 何心

院（系、所） 设计学院